Project Context: Group Tied Attention (GTA) for Edge-Optimized LLMs

1. Core Objective

The project focused on solving the memory bandwidth bottleneck during autoregressive LLM inference on resource-constrained edge hardware. The primary objective was to drastically reduce the VRAM footprint of the Key-Value (KV) cache without truncating the model, trading memory efficiency for deeper deductive reasoning.

2. Architectural Innovation: Group Tied Attention (GTA)

Standard Multi-Head Attention (MHA) allocates separate tensors for Keys (*K*) and Values (*V*) across all attention heads. GTA compresses this footprint through two simultaneous mechanisms:

- **Head Grouping:** Reducing the number of distinct KV heads (similar to Grouped Query Attention).
- **Tensor Tying:** Mathematically unifying the Key and Value matrices into a single, shared Answer tensor (*A*).

Instead of caching *K* and *V*, the autoregressive cache only stores *A*. This results in a theoretical memory reduction factor of $\frac{2 * H_{MHA}}{H_{GTA}}$

To implement this on-silicon without the compiler auto-allocating separate *K* and *V* tensors during the forward pass, the architecture relies on zero-copy view expansions using JAX's jnp.broadcast_to. This allows the unified *A* tensor to be projected into the attention mechanism without consuming additional VRAM.

3. The Positional Encoding Constraint (Why ALiBi over RoPE)

A critical mathematical constraint of tying *K* and *V* into a single *A* tensor is the incompatibility with Rotary Positional Embeddings (RoPE).

In modern architectures, *Q* and *K* act as routing logic, while *V* act as the semantic payload. RoPE applies a position-dependent geometric rotation matrix (R_m) to the tensors:

$$Output = \sum_{n=1}^{N} \alpha_{m,n} (R_n V_n)$$

If RoPE is applied to a unified *A* tensor, the payload is geometrically rotated based on its sequence position. This corrupts the semantic stability of the token (e.g., a token at position 5 points in a different dimensional direction than the same token at position 500). When the Feed-Forward Network (MLP) attempts to process this phase-shifted weighted sum, parameter capacity is wasted attempting to "un-rotate" the vectors, severely degrading validation perplexity.

The Solution: The architecture utilizes **Attention with Linear Biases (ALiBi)**.
ALiBi applies a static, distance-based scalar penalty ($m \cdot \Delta$) directly to the attention logits before the softmax operation, leaving the cached tensors entirely untouched:

$$Attention(Q, A, A) = softmax(\frac{Q \mathbin{@} A^T}{\sqrt{d}} + m \cdot \Delta)A$$

Because ALiBi requires zero positional rotation of the cache, the unified A tensor remains mathematically pure and semantically intact for use as the Value payload. Furthermore, the ALiBi mask is computed dynamically during the forward pass, requiring zero additional autoregressive cache memory.

4. Unviable Alternative RoPE Workarounds

During research, alternative RoPE implementations were mathematically ruled out due to edge-hardware constraints:

- **Learnable Un-rotation:** Storing rotated A tensors and using a learned weight matrix to un-rotate them for the Value payload is mathematically impossible, as RoPE is dynamic/position-dependent while learned weights are static.
- **On-the-Fly Rotation:** Storing unrotated A tensors and dynamically applying RoPE to generate temporary K tensors during the forward pass hits the "Autoregressive Compute Wall." It forces the system to pull the entire sequence history from the cache and execute heavy trigonometric math on every single step of generation, converting an $O(1)$ cache read into an $O(N)$ compute bottleneck.

5. Macro Strategy: Depth Reinvestment

The ultimate application of GTA is "Depth Reinvestment." By compressing the KV cache by up to 4x, the architecture avoids the need to truncate model layers to fit on edge GPUs. The saved VRAM is instead reinvested into adding more transformer blocks (e.g., scaling from 24L to 28L/30L). This achieves an "Iso-KV Cache" equilibrium where a much deeper, more intelligent model operates within the exact same memory constraints as a shallow, standard MHA model.